

ML-часть: пайплайн ИИ-агента по недвижимости

«Цифровой урбанист-детектив» — техническая спецификация зоны ответственности ML-инженера

Ключевой принцип: **своя модель не обучается**. Ценность системы — в оркестрации агентов и гибридном поиске (RAG) поверх заранее собранной базы, а не в весах модели. Всё, что похоже на «ML» — эмбединги, reranker, парсинг запроса — это **инференс готовых моделей**, а не обучение с нуля. Этот документ описывает то, что делает ML-инженер.

Две фазы системы

Прежде всего разделяем два режима работы — иначе задачи путаются:

- **OFFLINE** — сбор данных, гео-обогащение, генерация эмбедингов, укладка в БД. Делается заранее / по расписанию, батчем. Здесь вся вычислительная «тяжесть».
- **ONLINE** — оркестрация, гибридный retrieval, reranking, объяснение выдачи. Работает за секунды на запрос пользователя, в интернет не ходит.

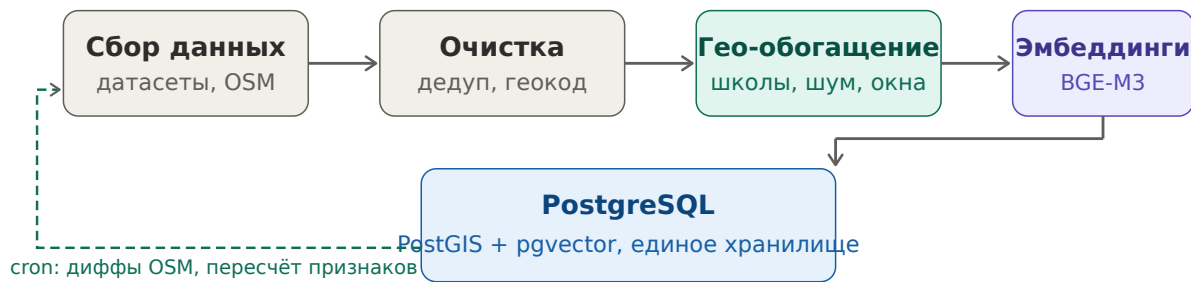
Именно эта граница отвечает на вопрос «парсить интернет на лету — долго ли»: весь тяжёлый парсинг и гео-расчёт живёт в offline-фазе, а онлайн-запрос обращается только к готовой базе.

Роли в команде (1 ML/Data + 2 Back).

ТЫ · ML/DATA — владеешь данными и моделями: от скрапинга до финального промпта. **BACK x2** — владеют инфраструктурой: БД как сервер, API-слой, интеграции внешних API, деплой.

Граница: ты отвечаешь за «что это значит и как модель поймёт», беки — за «как это надёжно посчитать, сохранить и отдать по сети».

Offline — подготовка (батчем, заранее)



Online — обработка запроса (в реальном времени)

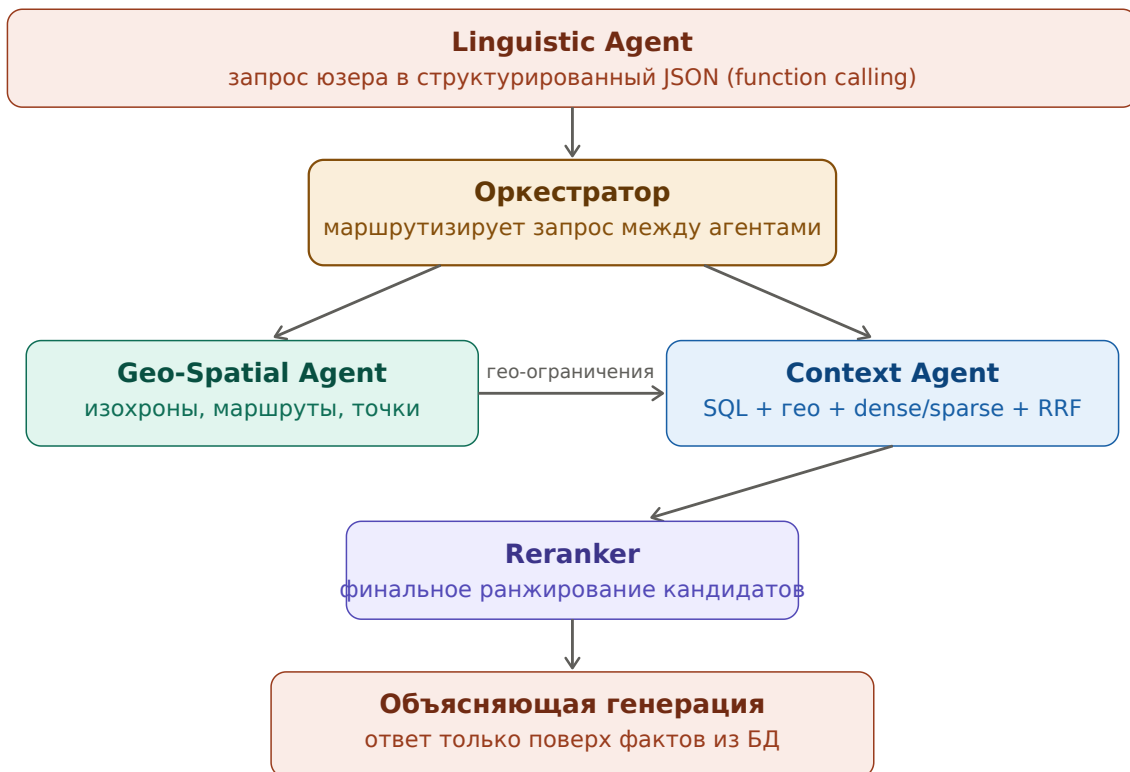


Схема потока: офлайн-подготовка данных (сверху) и онлайн-обработка запроса (снизу)

Фаза 1 — Offline: подготовка данных BATCH

1 Сбор сырых данных ТЫ · ML/DATA

Источники: готовые датасеты недвижимости РФ (Kaggle-выгрузки Циан/Домклик), OpenStreetMap для гео-объектов (школы, бары, алкомаркеты, парки, дороги), открытые рейтинги школ (ЕГЭ/олимпиады). Всё складывается в raw-таблицы как есть. Ты как дата-инженер команды пишешь скрапер и ETL целиком. Важно: датасет домов — это контент для базы, а не обучающая выборка.

2 Очистка и нормализация ТЫ · ML/DATA

Дедупликация объявлений, приведение адресов к единому виду, геокодирование (адрес → координаты через Nominatim/Yandex/DaData API), отсеив мусора. На выходе — чистая таблица `listings` со структурными полями: цена, площадь, этаж, координаты.

3 Гео-обогащение через PostGIS ТЫ · ML/DATA

Целиком твой шаг — и логика признаков, и сами PostGIS-запросы. Для каждого объекта заранее считаешь то, что дорого считать на лету, и пишешь результат отдельными колонками:

- пешая доступность до школ, метро, парков (изохроны);
- плотность баров и алкомаркетов в радиусе 500 м;
- ориентация окон по сторонам света + грубая инсоляция (азимут солнца против высотности соседних зданий);
- шумовые слои (близость ТТК/ЗСД/трамвайных путей).

Тогда online-запрос — это просто фильтр по готовому полю, а не расчёт в реальном времени.

4 Лёгкий классификатор признаков — опционально ТЫ · ML/DATA

Единственное место, где «обучение» вообще уместно: узкий классификатор на базе `ruBERT` по текстовым описаниям — для признаков, которых нет в структурных полях («скрытая коммуналка рядом», «двор-колодец»). Обучается на нескольких сотнях размеченных вручную примеров.

На хакатоне чаще заменяется few-shot промптом. Держи этот шаг как опцию, а не обязательный. Файн-тюн полноценной LLM не нужен: нет размеченного датасета пар «запрос → правильный дом», и за время хакатона он не появится.

5 Генерация эмбеддингов — инференс, не обучение ТЫ · ML/DATA

Текстовые описания ЖК и отзывы прогоняются через готовую многоязычную модель `BAAI/bge-m3` (сильна на кириллице, dim 1024, Apache 2.0). За один проход модель выдаёт **и dense-вектор, и sparse-представление** — оба пишутся в базу, это фундамент гибридного поиска. Минуты работы, никакого файн-тюна. Альтернатива при нехватке качества — `Qwen3-Embedding` (см. раздел «ML-стек»).

```
# псевдокод offline-инференса эмбеддингов
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-m3")

vectors = model.encode(descriptions, normalize_embeddings=True) # dim = 1024
# vectors -> колонка vector(1024) в Postgres
```

6 Укладка в единую БД ТЫ + BACK

Хранилище одно: **PostgreSQL + PostGIS + pgvector**. Структурные поля, гео-колонка (`geometry`), dense-вектор (`vector(1024)`) и sparse-вектор (`sparsevec` , `pgvector` ≥ 0.7) — в одной таблице, чтобы фильтры, гео и оба типа семантики решались в одной базе. Отдельная векторная БД (Chroma/Pinecone) не нужна: не умеет гео, размазывает данные по двум хранилищам.

Ты: проектируешь схему — какие поля нужны, размерность вектора под BGE-M3 (`vector(1024)` + `sparsevec`), тип и параметры индексов (HNSW на вектор, GiST на гео, B-tree на цену). Схема под *твоей* retrieval.

Беки: поднимают и держат сервер — установка Postgres, расширения PostGIS/pgvector, миграции, бэкапы, доступы. Ты диктуешь структуру — они накатывают и обслуживают.

7 Актуальность данных: инкрементальный цикл обновления ТЫ · ML/DATA

База — не разовый слепок, а живой слой города. Offline-фаза крутится по расписанию (cron), и изменения района попадают в систему без переиндексации и переобучения:

- **Гео-объекты:** OSM отдаёт *replication diffs* (поминутные/дневные диффы изменений) — через `pyosmium` забираем только то, что изменилось, а не весь город заново. Новый магазин/бар → `upsert` в таблицу POI.
- **Объявления:** повторный прогон источников, `upsert` по внешнему `id`; пропавшие из выдачи помечаются `is_active = false` — проданное не всплывает в поиске, но история остаётся.
- **Каскадный пересчёт признаков:** ключевой момент — новый POI меняет не только свою строку, но и производные колонки квартир вокруг. `ST_DWithin(geom, new_poi, 500)` находит затронутые listings (десятки, не вся база), и только у них пересчитываются `bar_density_500m`, шумность и т.п. Секунды работы.
- **Эмбединги — только по изменившимся текстам:** хэш описания в отдельной колонке; описание не менялось → вектор валиден, не трогаем. Реиндекс не нужен.
- **Freshness наружу:** колонка `updated_at` у каждой сущности → система честно показывает «данные актуальны на ...» в ответе.

Эффектный момент демо: добавить магазин в базу руками при жюри — выдача меняется на следующем же запросе. ~20 строк кода, а демонстрирует главный аргумент архитектуры: знание живёт в БД, а не в весах. «Бар открылся вечером → ночью `cron` подтянул → утром система отсекает квартиры рядом». В архитектуре с обучением это стоило бы переобучения.

Фаза 2 — Online: обработка запроса REAL-TIME

1 Linguistic Agent — разбор запроса (NLU) ТЫ · ML/DATA

Свободный текст пользователя → структурированный JSON через function calling (Qwen через OpenRouter, temperature=0, Pydantic-валидация с retry — см. раздел «Надёжность»). ML-инженер проектирует как модель извлекает поля: пишет промпт и схему, гоняет на примерах запросов, ловит краевые случаи («компромисс между двумя работами» ломает наивную схему).

```
# «тихо, юго-запад, школа рядом, без баров, бюджет бизнес» превращается в:
{
  "price_max": 25000000,
  "geo_constraints": {"near_school": true, "walk_minutes": 15},
  "window_orientation": ["SW", "W"],
  "noise_max": "low",
  "stop_factors": ["bars", "communal_flats"]
}
```

2 Оркестратор ТЫ + BACK

Смотрит на распарсенный запрос и решает, каких агентов и в каком порядке вызвать — здесь «интеллект» системы. Обычный запрос → сразу фильтры + вектор. «Компромисс Сколково ↔ Сити» → сначала Geo-агент считает центральную точку, потом фильтры.

Ты: логика маршрутизации — промпт и схема function calling, описания инструментов (`get_isochrone` , `search_db`), правила ветвления. Это «мозг».

Беки: сервис-обёртка — надёжные вызовы LLM API, retry/таймауты, передача данных между шагами, склейка в единый бэкэнд. Это «водопровод».

Реализация логики: **function calling на одной LLM** достаточно для демо. **LangGraph** — как «сюда масштабируемся», если нужны циклы («не нашли за 15 мин → расширь до 20 → перезапусти»).

3 Geo-Spatial Agent BACK + ТЫ

Изохроны, маршруты, центральные точки через OSM + OpenRouteService / Valhalla.

Беки: клиент к внешнему гео-API — вызовы OpenRouteService/Valhalla, обработка ошибок, кэширование маршрутов. Ты говоришь «нужна изохрона 15 мин пешком от точки» — они реализуют вызов и отдадут результат.

Ты: определяешь, какие изохроны/радиусы нужны и как превратить их в фильтр для retrieval — совместить гео-ограничение с векторным поиском.

4 Context Analytic Agent — гибридный retrieval (сердце RAG) ТЫ · ML/DATA

Запрос в Postgres задействует каналы: структурный фильтр отсекает большинство базы, гео-фильтр проверяет достижимость, векторный поиск ранжирует по смыслу. Четвёртый канал — **sparse-вектор** BGE-M3 (тип `sparsevec` в `pgvector` ≥ 0.7): лексический матчинг для точных терминов («школа 239», «ЖК Символ»), которые `dense` размывает. `Dense`- и `sparse`-ранжирования сливаются через **RRF** до реранка.

```
SELECT *, embedding <=> :query_vec AS distance
FROM listings
WHERE price <= 25000000 -- структурный фильтр
  AND ST_DWithin(geom, :school_point, 1200) -- geo (PostGIS)
ORDER BY embedding <=> :query_vec -- dense (pgvector)
LIMIT 50;
-- аналогичный запрос по sparse_embedding, затем RRF-слияние двух списков
```

Грабля фильтрованного HNSW: при жёстких `WHERE`-фильтрах HNSW-индекс может вернуть меньше `LIMIT` (индекс отдаёт `top-K` до фильтра, фильтр их выкашивает). Лечится включением `SET hnsw.iterative_scan = strict_order` (`pgvector` ≥ 0.8) — индекс досканирует, пока не наберёт `LIMIT`. Эксперт на защите это спросит.

5 Reranker ТЫ · ML/DATA

Модель `bge-reranker-v2-m3` (мультиязычная, работает с русским) причёсывает топ-кандидатов в осмысленный финальный порядок. Готовая модель, без обучения. Именно здесь видно «интеллект» ранжирования.

6 Объясняющая генерация ТЫ · ML/DATA

LLM формулирует ответ **только поверх фактов из БД**: «подходит — школа 239 в 12 минутах пешком без крупных проспектов, окна ЮЗ, баров в радиусе 500 м нет». Генерация не выдумывает адреса → ноль галлюцинаций.

Разделение ответственности — ты vs два бэкендера

Кто что делает руками. На стыковых шагах — конкретно, где проходит граница:

Шаг	Ты · ML/Data	Беки ×2
Сбор данных, скрапинг, ETL	целиком твоё	—
Очистка, геокодирование	целиком твоё	—
Гео-обогащение (PostGIS)	логика признаков + запросы	—
Классификатор ruBERT (опц.)	целиком твоё	—
Эмбединги (BGE-M3, dense+sparse)	целиком твоё	—
База данных	схема, индексы, поля под поиск	сервер, расширения, миграции, бэкапы
Linguistic Agent (NLU)	промпт, схема, извлечение	—
Оркестратор	логика ветвления, промpty	сервис-обёртка, вызовы API, retry
Geo-Spatial Agent	какие изохроны, как в фильтр	клиент к ORS/Valhalla, кэш
Гибридный retrieval-запрос	целиком твоё	—
Reranker	целиком твоё	—
Объясняющая генерация	целиком твоё	—
API-слой, деплой, фронт-интеграция	—	целиком их

Граница одной фразой: ты владеешь всем, что про данные и модели — от скрапинга до финального промпта. Беки владеют тем, что превращает твой пайплайн в работающий сервис — БД как инфраструктура, API, интеграции, деплой.

ML-стек: конкретные решения (актуально на 2026)

Что реально брать сегодня. Все модели — готовые, ставятся через `sentence-transformers` / HuggingFace, обучать не нужно. Одна оговорка: **смена embedding-модели = полный реиндекс базы**, поэтому выбор фиксируется на старте.

Слой	Решение	Почему / альтернатива
Эмбединги	<code>BAAI/bge-m3</code> хакатон	Рабочая лошадка мультязычного RAG, сильна на кириллице, dim 1024, Apache 2.0, self-host. Даёт dense + sparse из одной модели.
	<code>Qwen3-Embedding</code> (0.6B / 4B) качество	#1 на MTEB multilingual, гибкая размерность, instruction-aware. Тяжелее — берётся, если BGE-M3 не хватает по качеству.
Reranker	<code>BAAI/bge-reranker-v2-m3</code> хакатон	Безопасный дефолт: мультязычный кросс-энкодер, +5...15 NDCG@10 к выдаче. Хорошо обкатан.
	<code>Qwen3-Reranker</code> (0.6B) качество	Сильнее в instruction-following (можно объяснить модели, что значит «релевантно»). Апгрейд, если реранк — узкое место.
LLM (парсинг + оркестрация + объяснение)	Qwen через OpenRouter хакатон fallback: DeepSeek → GPT-4o-mini / Gemini Flash	Один ключ, нативный фоллбек (массив <code>models</code> в запросе). Primary — лучший русский + tool calling; фоллбеки — другие провайдеры, чтобы не лечь вместе. Temperature=0 для парсинга.
Классификатор признаков (опц.)	<code>ruBERT</code> (fine-tune)	Только если нужны признаки из текста, которых нет в структуре. Иначе — few-shot промпт.
Векторное хранилище	PostgreSQL + pgvector (HNSW)	Единое хранилище с гео (PostGIS) и фильтрами. Отдельная векторная БД не нужна на этом объёме.
Сервинг ML	FastAPI (Python) хакатон	ML-first экосистема (PyTorch, HF, LangChain) — Python-only.

Слой	Решение	Почему / альтернатива
		Автодоки OpenAPI ускоряют стыковку с бекками.
	+ TEI для эмбедингов/перанка прод	Text Embeddings Inference: лучше батчинг и ONNX-ускорение, чем голый sentence-transformers.

Правило выбора: **хакатон** — берём для демо; **качество/прод** — апгрейд «после», если конкретный слой окажется узким. Не тратить дни на выбор «идеальной» модели — стартовать на BGE-M3, мерить на реальных запросах, менять точно.

Надёжность и качество: что отличает призовую ML-часть

Модели из стека — это скелет. Ниже то, что превращает «прикрутили LLM к базе» в спроектированную систему. Отсортировано по соотношению «эффект / трудозатраты»:

1 Eval-харнесс (golden set) — главное

30–50 эталонных запросов: «запрос → правильный JSON-парс» и «запрос → ожидаемые объекты в топе». Метрики: точность парсинга, recall@10, NDCG@10 после реранка. Даёт выбор моделей по цифрам, защиту от регрессий при правке промптов и — **метрики на слайде защиты**, что жюри ценит на порядок выше «работает хорошо». Полдня работы, максимальный ROI.

2 Гибридный поиск: dense + sparse + RRF

Механика описана в online-шаге 4. Здесь суть, почему это must-have именно для вас: домен перенасыщен именами собственными (номера школ, названия ЖК, аббревиатуры магистралей) — на них dense-поиск систематически проседает, и без sparse-канала «школа 239» найдёт «сильные школы вообще». Замерь на golden set поиск с RRF и без — разница станет слайдом защиты.

3 Валидация и самопочинка выходов LLM

Парсинг: Pydantic-схема, temperature=0, retry-петля — невалидный JSON отправляется модели обратно вместе с текстом ошибки, она чинит (2–3 попытки). Превращает «иногда ломается» в «практически никогда».

4 Relaxation loop в оркестраторе

«Не нашли за 15 мин пешком → расширь до 20 → перезапусти поиск». На демо жюри введёт запрос, под который база пуста — система, честно сообщающая «расширил критерий, вот что есть», выглядит умной, а не сломанной. Аргумент за LangGraph.

5 Трейсинг пайплайна (Langfuse / Phoenix, self-host)

Каждый запрос логируется по шагам: парс → SQL → retrieval → rerank → генерация. Без этого дебаг 6-шагового пайплайна — вслепую; с этим — минута до места поломки. Ставится за час.

6 Мультиязычность — дёшево, в тему международного жюри

BGE-M3 (100+ языков) и Qwen уже мультиязычны — русский захардкожен только в промптах. Сделать их языконезависимыми — и английский запрос «quiet flat near a strong math school» работает поверх русской базы. Кросс-языковой поиск — эффектный момент демо почти бесплатно.

7 Кэш + «демо-режим»

Кэш эмбедингов запросов и ответов LLM по хэшу. Повторные запросы летают, а сценарий защиты **не зависит от того, жив ли OpenRouter в момент демо**: golden-запросы прогнаны и закэшированы заранее, живые вопросы жюри идут полным пайплайном.

8 Деградация по слоям

Фоллбек не только у LLM: вектор недоступен → SQL+гео фильтры (хуже, но живо); reranker упал → отдаём порядок retrieval. Каждый слой умеет отвалиться, не убив систему.

Связь ML ↔ бэкэнд (Go): кто с кем говорит

Бэк на Go, ML на Python — это **нормальный полиглот-паттерн**, а не проблема. Ключевой факт, снимающий вопрос «может, всё на Go»: узкое место — инференс модели (50–200 мс), а не обработка HTTP (~0.5 мс). Переписывать ML на Go бессмысленно — выигрыш микросекунды на запросе в сотни миллисекунд. Каждый язык делает своё.

Go-бэк (беки x2), фреймворк — Fiber: API-шлюз для фронта, аутентификация, роутинг, оркестрация вызовов, работа с внешними гео-API (ORS/Valhalla), высоконагруженный трафик. Latency-critical слой. Fiber (на fasthttp) — быстрый роутинг и клиент.

Python ML-сервис (ты): FastAPI-эндпоинты, за которыми живут модели — эмбединги, reranker, вызовы LLM, retrieval-логика. Всё, что грузит веса и думает.

Протокол между ними

Бюджет латентности одного запроса: эмбединг ~10–30 мс + retrieval ~10–50 мс + rerank ~50–150 мс + LLM ~500–3000 мс. Транспорт (<5 мс) — не узкое место; выбор протокола не влияет на скорость системы.

Вариант	Когда	Вердикт
REST / JSON (FastAPI ↔ Fiber)	Go (Fiber-клиент) дёргает Python по HTTP, тело — JSON. Автодоки Swagger, читаемо, дебажится через curl. Протокол не меняется — Fiber лишь фреймворк на стороне Go.	Берём на хакатон. Проще, быстрее поднять, на нашем объёме разница незаметна.
gRPC (protobuf)	Общий <code>.proto</code> -контракт генерит типизированные стабы в Go и Python. Payload в 3–10× меньше, ниже p99 на высоком RPS. gRPC в Go — отдельная библиотека, с Fiber уживается (Fiber остаётся на фронтном API).	Апгрейд «сюда масштабируемся»: когда ML-сервис дёргают часто и латентность каждого хопа важна.

Что ты отдаёшь беку (контракт стыка). Твой ML-сервис принимает сырой запрос пользователя и возвращает *готовый ранжированный список объектов + объяснение*. Внутри — весь пайплайн (парсинг → retrieval → rerank → генерация). Для бэка это чёрный ящик: `POST /search {query, filters?} → {results: [...], explanation}`. Гео-вызовы (ORS/Valhalla) может делать либо твой сервис, либо Go — договоритесь на старте; проще, если гео-API дёргает Go и передаёт тебе готовые изохроны.

Ключевые тезисы для защиты

1. Обучения нет на критическом пути. Эмбединги (BGE-M3), reranker (bge/Qwen3), парсинг (LLM) — инференс готовых моделей. Единственная опциональная обученная штука — узкий ruBERT-классификатор, и он не обязателен.

2. Граница offline/online = ответ на «парсить долго». Весь тяжёлый парсинг и георасчёт заранее, батчем. Онлайн-запрос ходит только в свою БД → отвечает за секунды.

3. Интеллект системы = оркестратор + гибридный RAG, а не веса модели. Это отличает проект от «прикрутили GPT к базе» и читается как инженерная зрелость.

4. Данные живут вместе с городом. Инкрементальный цикл (OSM-диффы → upsert POI → каскадный пересчёт признаков только у затронутых квартир) держит базу актуальной без переиндексации и переобучения. Новый бар на районе виден системе со следующего запроса — прямое следствие выбора «знание в БД, а не в весах».

Стек ML-части одной строкой: BGE-M3 (dense+sparse эмбединги) → PostgreSQL + PostGIS + pgvector (единое хранилище + гибридный поиск с RRF) → bge-reranker-v2-m3 (ранжирование) → Qwen через OpenRouter с фоллбек-цепочкой DeepSeek → GPT-4o-mini (парсинг + оркестрация + объяснение), обернуто в FastAPI и связано с Go-бэком (Fiber) по REST. Ноль обучения, ноль парсинга на лету, метрики на golden set.